Multimedia System Homework

Student information

Name	: Richard Rafer Guy
Student ID	: M11509801

The program was developed and tested on Windows using Visual Studio Code. All commands were executed through the integrated terminal in Visual Studio Code. Python 3.10.11 was used as the programming language, with a virtual environment (`venv`) created to isolate the required dependencies. The required Python packages were installed from `requirements.txt`. The program can be run from the project root directory after activating the virtual environment and executing `python main.py`. No special environment variables are required for this program.

To run the program from a clean environment:

1. Open the project folder in Visual Studio Code.
2. Open the integrated terminal.
3. Create a virtual environment: python -m venv .venv
4. Activate the virtual environment: `.venv\Scripts\activate`
5. Install the required dependencies: `pip install -r requirements.txt`
6. No special environment variables are required for this program. The program runs using the installed Python environment and the required Python packages.

Running the Program

The program can be run from the project root directory using:

```
python main.py <option>
```

The available options are:

- python main.py part1 — runs all experiments and outputs for Part 1.
- python main.py part2 — runs all experiments and outputs for Part 2.
- python main.py part3 — runs all experiments and outputs for Part 3.
- python main.py part4 — runs all experiments and outputs for Part 4.
- python main.py all — runs Parts 1, 2, 3, and 4 sequentially.

Therefore, <option> should be replaced with part1, part2, part3, part4, or all, depending on which part should be executed.

Output Location

The generated results are saved in the outputs directory. Each assignment part has its own subfolder:

- outputs/part1/ — contains the generated results for Part 1.
- outputs/part2/ — contains the generated results for Part 2.
- outputs/part3/ — contains the generated results for Part 3.
- outputs/part4/ — contains the generated results for Part 4.

For example, running python main.py part3 generates the Part 3 outputs in outputs/part3/, while running python main.py all generates the outputs for all parts in their corresponding folders. All figures included in this report were generated from the program outputs stored in these corresponding output folders.

Part 1 : Two Roads to the Same Curve

The following section presents the results of my work for Part 1, including the implementation results, generated figures, measurements, and the required analysis for each point.

Point 1: Implement both. Choose four control points that give a clearly curved shape spanning most of your canvas, and show the curve each construction produces (one overlaid figure is enough, at a shallow depth where the two are still visibly different) (1 point).

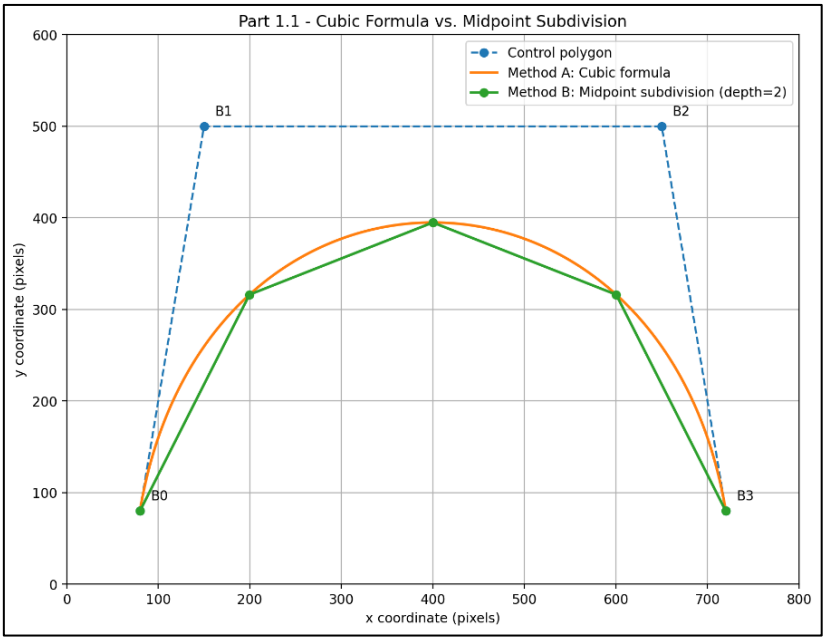


Figure 1: Cubic Bézier curve generated using the direct formula and midpoint subdivision at depth 2, using the same four control points.

For this experiment, I implemented the Bézier curve using two different constructions. **Method A** evaluates the cubic Bézier formula directly, while **Method B** uses recursive midpoint subdivision.

The same four control points were used for both methods:

- Control Point 0 = (80,80)
- Control Point 1 = (150,500)
- Control Point 2 = (650,500)
- Control Point 3 = (720,80)

An 800 × 600 coordinate space was used, with one coordinate unit treated as one pixel for the measurements.

For **Method A**, the curve coordinates were computed using the cubic Bézier formula $C(t) = (1-t)^3 b_0 + 3t(1-t)^2 b_1 + 3t^2(1-t) b_2 + t^3 b_3$ and 1000 parameter samples were used to display the curve.

For **Method B**, recursive midpoint subdivision was used with a subdivision depth of 2. At this depth, the construction produces (2^2=4) line segments and (2^2+1=5) polyline vertices.

The figure above overlays the two results. The cubic-formula curve is shown as the smooth curve, while the midpoint construction is shown as a polyline. At this shallow subdivision depth, the

difference between the two constructions is still clearly visible, especially between consecutive polyline vertices.

Point 2: Convergence. Overlay B on A. Measure the gap this way: for every point on the formula curve, take its distance to the nearest segment of your polyline; report the largest — in that direction, not the reverse. Sample A finely enough — consecutive points under 0.1 px apart — or you are measuring your own sampling of A, not the gap. Report it as a function of subdivision depth, then raise the depth until it falls below 0.5 px and report that depth. State the rule your errors follow each time the depth increases by one, and whether the shallow depths obey it as closely as the deep ones. Finally: the formula makes you decide in advance how many values of t to sample. Name one thing the mid-point construction lets you do that this does not. (1.5 points)

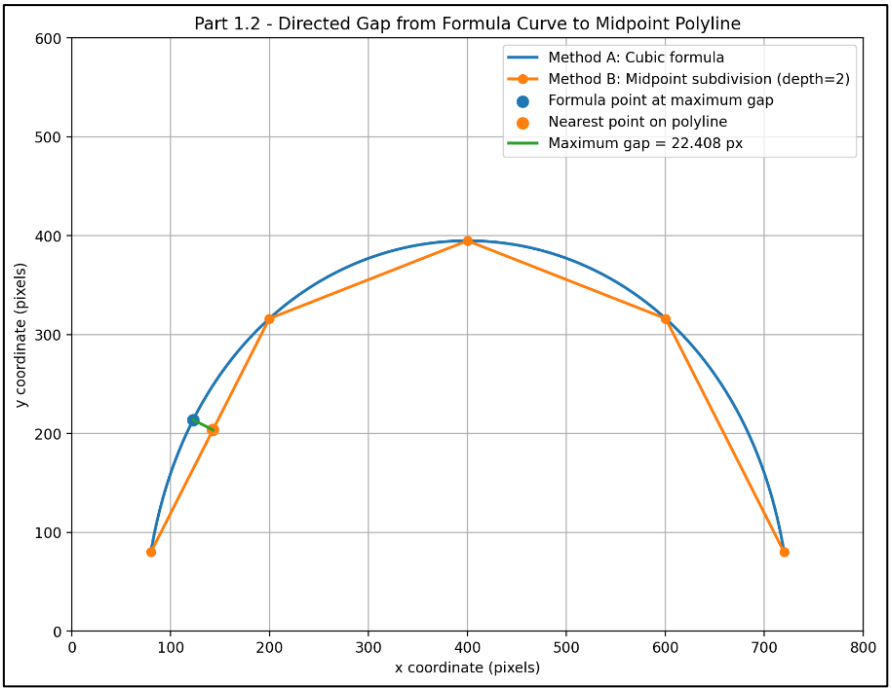


Figure 2: Directed gap from the formula curve to the midpoint-subdivision polyline at subdivision depth 2.

To measure convergence, Method B was overlaid on Method A. For every sampled point on the formula curve, I measured its distance to the nearest segment of the midpoint-subdivision polyline. The smallest distance to the polyline was found for each formula-curve point, and the largest of these distances was reported as the maximum gap. The measurement was performed only in the direction from the formula curve to the polyline, not in reverse.

At subdivision depth 2, the measured maximum gap was approximately 22.408 px. The figure 2 also shows the formula-curve point where this maximum gap occurs and its nearest point on the polyline.

The formula curve was sampled densely enough to ensure that the distance between every pair of consecutive sampled points was below 0.1 px. The final sampling used 16,385 points, with a measured maximum consecutive spacing of 0.077961 px. Therefore, the sampling condition required by the assignment was satisfied.

The maximum gap was then measured for increasing subdivision depths. The numerical results generated by the program were:

```
Convergence results
-------------------
Depth    Segments    Max Error (px)    Reduction
    0           1        315.000000            -
    1           2         84.662887     3.720639
    2           4         22.407697     3.778295
    3           8          5.714945     3.920894
    4          16          1.438456     3.972972
    5          32          0.360525     3.989893
    6          64          0.090225     3.995833
    7         128          0.022567     3.998110
```

Figure 3: Convergence results showing the maximum gap and error-reduction factor for subdivision depths 0 through 7.

The first subdivision depth at which the maximum gap became smaller than 0.5 px was depth 5, where the measured maximum gap was 0.360525 px.

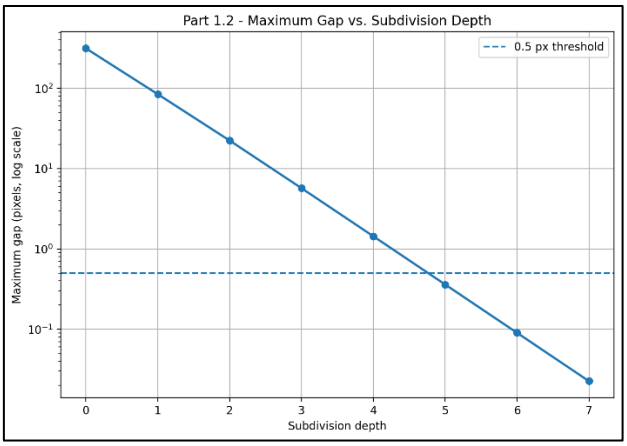


Figure 4: Maximum gap as a function of subdivision depth. The dashed line represents the 0.5 px threshold.

The error reduction factors between consecutive subdivision depths were approximately 3.720, 3.778, 3.921, 3.973, 3.990, 3.996, and 3.998. Therefore, as the subdivision depth increases by one, the maximum error approaches a pattern in which it becomes approximately one quarter of the previous error.

The shallow subdivision depths already follow the same general trend, but they do not follow it as closely as the deeper depths. At larger depths, the reduction factor becomes increasingly close to 4, indicating a more regular convergence pattern.

Finally, one advantage of the mid-point construction is that the curve can be refined recursively where more detail is needed. If a part of the curve is already approximated accurately enough, subdivision can stop there, while a more strongly curved part can be subdivided further. In contrast, direct formula sampling requires a fixed number of parameter values (t) to be chosen in advance for the whole curve.

Point 3: Joins. Build an open path of three Bézier segments; that gives you two joins. Make one smooth and one a corner point, and say in one line what makes the corner a corner, in terms of its direction lines. Chapter 4 states the condition for a smooth join as “control points are lined up and direction lines are of the same length”. Isolate one half of that: keep the join point and the two direction points on either side of it collinear, but make the two direction lines clearly unequal in length. Show it next to the equal-

length version and say whether the unequal-length join still looks smooth. If it does, explain what the equal-length half buys you, given that the shape does not need it. (1.5 points)

I constructed an open path consisting of three cubic Bézier segments. Since the three segments are connected in sequence, the path contains two joins. The first join was designed to be smooth, while the second join was designed to form a corner.

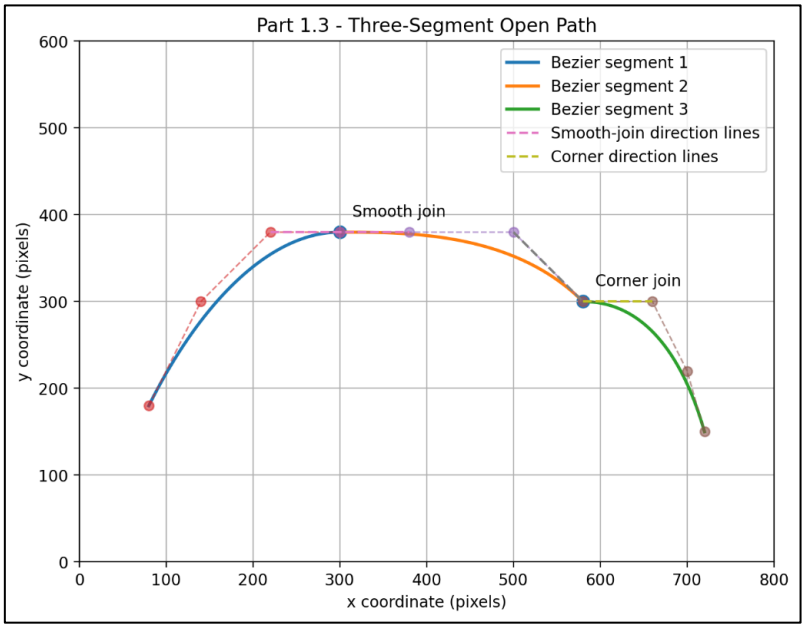


Figure 5: Open path consisting of three Bézier segments with one smooth join and one corner join.

At the smooth join, the direction points on both sides of the join are lined up with the join point, so the two segments continue in the same direction. At the corner join, the direction lines of the two adjacent Bézier segments are not lined up at the join. Therefore, the direction changes abruptly and a visible corner is formed.

To examine the role of direction-line length separately, I created another comparison in which the join point and the two direction points on either side remain collinear. In the first version, the two direction lines have equal lengths. In the second version, they remain collinear but their lengths are clearly unequal. Therefore, the direction is kept the same while only the direction-line length is changed.

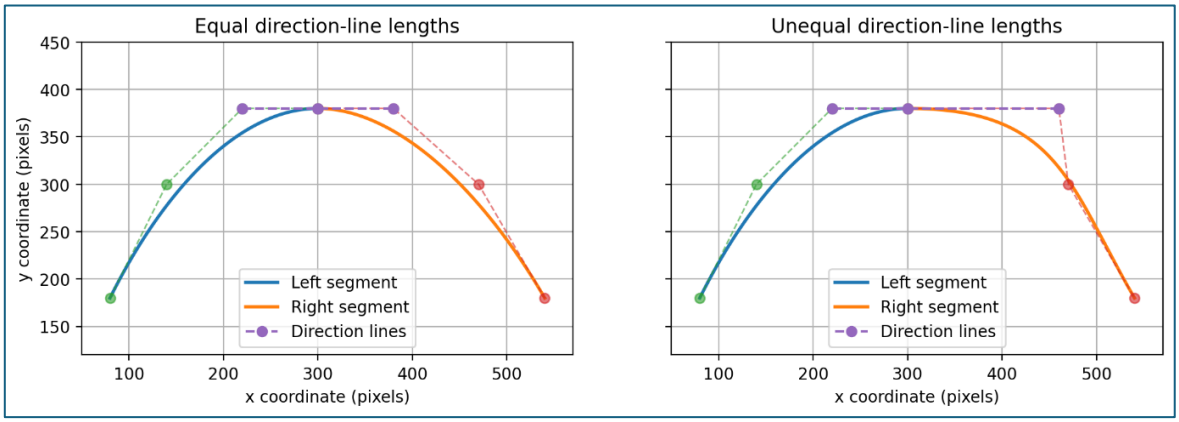


Figure 6: Comparison of equal-length and unequal-length direction lines while keeping the direction points collinear.

The unequal-length version still appears visually smooth because the direction lines remain collinear, so there is no abrupt change in direction at the join. This shows that equal direction-line lengths are not necessary for the geometric shape itself to appear smooth.

However, equal direction-line lengths provide an additional property. For a cubic Bézier curve, the length of the direction line affects how fast the curve progresses as the parameter changes near the join. If the direction lines on both sides have the same length and direction, the two Bézier segments have the same parametric speed at the join. If their lengths are different, the join can still look smooth because the directions are still aligned, but the two segments progress at different speeds.

Part 2 : Inside and Outside

The following section presents the results of my work for Part 2, including the implementation results, generated figures, and the required analysis for each point.

Point 1 : Implement a scanline fill for a closed path using the non-zero winding number rule, as Chapter 4 defines it (its "line" must be read as a ray — a full line would count the crossings on both sides): draw a ray from the point you are testing in any direction; add one each time the path crosses it from left to right, subtract one each time it crosses from right to left; non-zero means inside. Scanline means you work one row of pixels at a time instead of one pixel at a time: find where the path crosses the row, then walk along the row accumulating the signed crossings, and fill wherever the running total is non-zero. That running total is the winding number, so "crosses from left to right" becomes a statement about whether an edge passes through the row upwards or downwards — settle once which of the two is +1 in your coordinate system, and keep it. Filling alternate spans between crossings is a different rule — that is even-odd — and on a self-intersecting path it gives a different picture; item 2 is such a path. Your fill must work for a path built from more than one sub path. (3 points)

The fill algorithm uses the non-zero winding number rule. For each horizontal scanline, the algorithm starts with a winding number of zero and processes the path crossings from left to right. Each crossing contributes either +1 or -1 according to the direction of the edge. A pixel is classified as inside when the accumulated winding number is non-zero and is therefore assigned the fill color. A winding number of zero indicates that the pixel is outside the path.

The renderer uses a screen-coordinate system with the origin at the top-left, x increasing to the right, and y increasing downward. Scanlines are processed from left to right. I use a downward edge crossing, from smaller y to larger y, as +1 and an upward edge crossing as -1. Horizontal edges do not contribute to the winding number. A half-open vertical interval is used when testing an edge against a scanline so that a vertex shared by two edges is counted only once. This convention is used consistently throughout Part 2.

This implementation does not simply alternate between filled and unfilled spans after every crossing, because that would implement the even-odd rule. Instead, it maintains the signed winding number across the complete scanline. This distinction becomes important for self-intersecting paths, such as the five-pointed star used in Point 2, where the center can have a winding magnitude greater than one.

The implementation also supports paths containing multiple sub-paths. The edges from every sub-path are collected and processed together on each scanline, so all of their signed crossings contribute to the same winding calculation.

The following are the specifications and conventions used in my implementation:

- **Coordinate origin:** Top-left
- **Positive x direction:** Right
- **Positive y direction:** Down
- **Scan direction:** Left to right
- **Downward crossing:** +1
- **Upward crossing:** -1
- **Horizontal edge:** Ignored
- **Inside condition:** winding != 0
- **Shared vertex handling:** Half-open interval
- **Crossing position:** Linear interpolation
- **Multiple sub-paths:** Supported

The visual results of this implementation are presented in the discussion of the following points.

Point 2 : Draw a five-pointed star as a single self-intersecting closed path — five vertices visited in "every other one" order, not ten alternating inner and outer vertices (Figure 2, left). Fill it. Show the result and give the winding number at the center — its sign follows from the convention you settled on in item 1, and either sign is acceptable. (1 point)

Five-Pointed Star Construction

I constructed the five-pointed star as a single self-intersecting closed path using exactly five vertices. The five vertices were visited in an "every other one" order, rather than using ten alternating inner and outer vertices. The visiting order was P0 → P1 → P2 → P3 → P4 → P0, where P0 is the bottom vertex, P1 is the upper-right vertex, P2 is the lower-left vertex, P3 is the lower-right vertex, and P4 is the upper-left vertex. The path was rendered on an 800 × 800 pixel canvas, with the star centered at (400, 400) and using a radius of 290 pixels. Each edge of the closed path was rasterized manually using the Bresenham line algorithm.

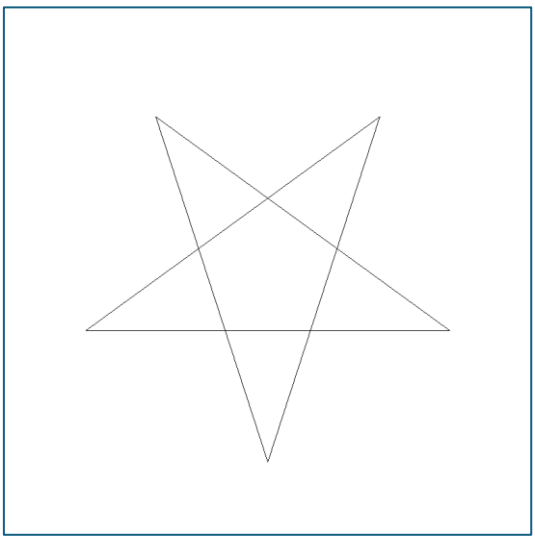


Figure 7: Five-pointed star constructed as a single self-intersecting closed path using five vertices visited in an "every other one" order.

Filling the Star

The five-pointed star was then filled using the scanline fill with the non-zero winding number rule implemented in Point 1. For each row, the path crossings were processed from left to right and their signed contributions were accumulated into the winding number. Pixels were filled whenever the running winding number was non-zero. The final rendering used black for the path boundary, RGB (70, 130, 200) for the filled area, and white for the background.

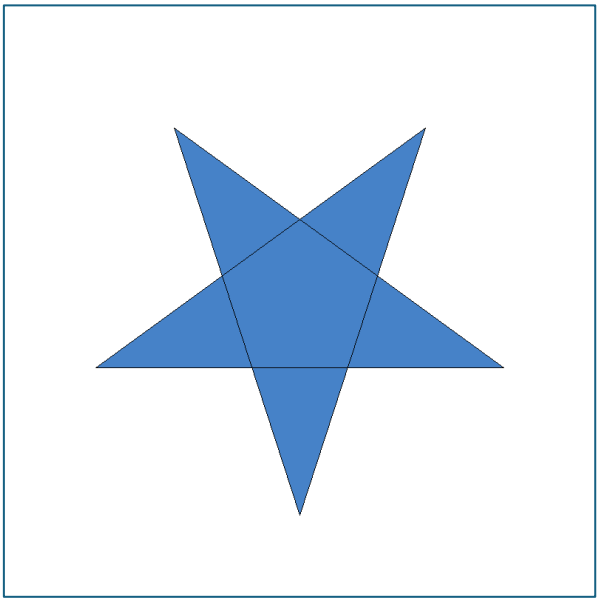


Figure 8: Filled five-pointed star using the scanline fill with the non-zero winding number rule.

Winding Number at the Center

Using the convention settled on in Point 1, where a downward crossing contributes +1 and an upward crossing contributes −1, the **winding number at the center of the star was +2**. Since +2 is non-zero, **the center is classified as inside the path and is therefore filled**. The sign of the winding number follows from the convention used in Point 1 and the direction in which the closed path is traversed.

Point 3 : Chapter 4 ends the winding rule with one sentence: "Depends on the path's having a direction." Make that sentence visible. Build a path from two sub-paths, an outer circle and a smaller inner circle (Figure 2, right). Describe each circle as four Bézier segments — the two direction points sit 0.5523·r along the tangent from each endpoint — and flatten them with your Part 1 construction; that is the whole pipeline, end to end. Render it twice, changing nothing but the direction in which the inner circle is traversed — not one point may move. Show both results, give the winding number at the center in each case, and explain the difference. Then say what this implies about how the letter "O" has to be stored in a font file. (1 point)

In this implementation, the direction in which a sub-path is traversed determines the sign of its contribution to the winding number. Therefore, changing the direction of one sub-path can change the winding number and the final fill result, even when no point coordinates are moved.

I built one path from two sub-paths, consisting of an outer circle and a smaller inner circle. Both sub-paths use the same center at (400, 400) on an 800 × 800 pixel canvas. The outer circle uses a radius of 290 pixels, while the inner circle uses a radius of 145 pixels.

Each circle was described as four Bézier segments. For every Bézier segment, the two direction points were placed 0.5523·r along the tangent from each endpoint, where r is the radius of the corresponding circle. The four Bézier segments were then flattened using the Part 1 construction with a subdivision depth of 6. This produced 256 flattened points for the outer circle and 256 flattened points for the inner circle. The flattened line segments were rasterized manually using the Bresenham line algorithm before the scanline fill was applied, completing the pipeline end to end.

The path was rendered twice, changing nothing but the direction in which the inner circle was traversed. In the first rendering, the outer circle and the inner circle were traversed in the same direction. In the second rendering, only the direction of the inner circle was reversed. No point was moved; only the visiting order of the inner circle was reversed.

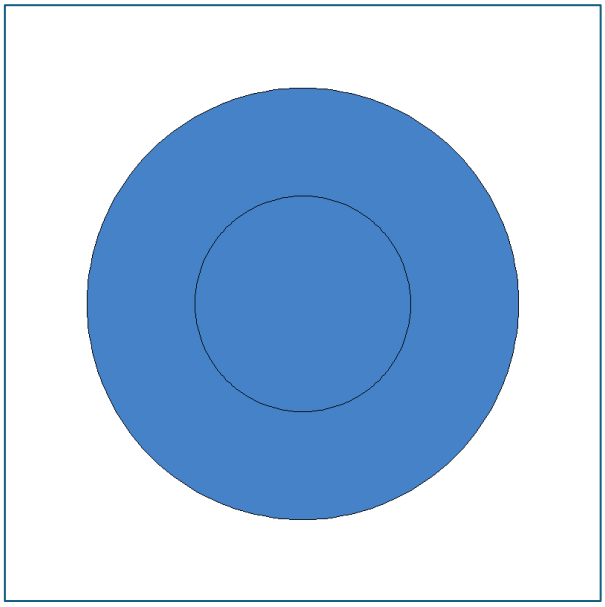


Figure 9: Path with the outer circle and the smaller inner circle traversed in the same direction.

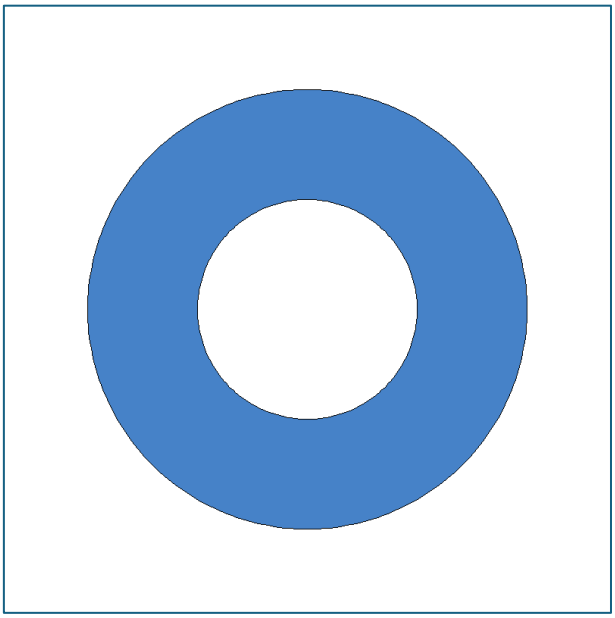


Figure 10: The same path with only the direction of the inner circle reversed; no point position was changed.

In the first rendering, where the outer circle and the inner circle were traversed in the same direction, the winding number at the center was −2. Since this value is non-zero, the center was classified as inside and was filled.

In the second rendering, where only the direction of the inner circle was reversed, the winding number at the center was 0. In this case, the winding contributions from the outer circle and the inner circle canceled at the center. Since the resulting winding number was zero, the center was classified as outside and remained unfilled.

This implies that, when the non-zero winding number rule is used, the outer and inner contours of the letter "O" must be stored with appropriate path directions. The inner contour must contribute an opposite winding direction relative to the outer contour so that the winding number inside the center region becomes zero. This allows the center of the letter "O" to remain a hole instead of being filled.

Part 3 : Landing on the Pixel Grid

The following section presents the results of my work for Part 2, including the implementation results, generated figures, and the required analysis for each point.

I. Area Coverage (2 points)

Point I.a: Rotate your star from Part 2.2 by a few degrees — 5° is enough — so that no edge is exactly horizontal or vertical, and use that rotated star wherever Part 3 asks for the star (I and II; III uses a different shape). A regular five-pointed star drawn as in Figure 2 has one exactly horizontal edge, and an axis-aligned edge is the one case where the staircase never appears at all.

The five-pointed star from Part 2 was rotated by 5° about its center at (200, 200) on a 400 × 400 pixel canvas. The star used a radius of 200 pixels. The rotation was applied so that no edge was exactly horizontal or vertical. After the rotation, the program checked all edges of the star and obtained Axis-aligned edge present: False, confirming that no edge was exactly horizontal or vertical.

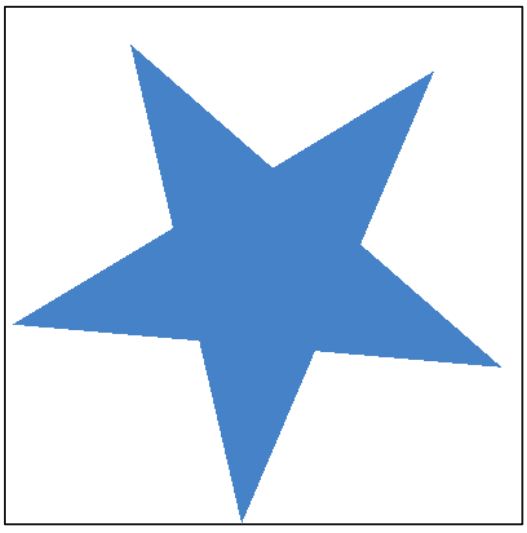


Figure 11: Five-pointed star from Part 2 after a 5° rotation. No edge is exactly horizontal or vertical.

The star from Part 2 was rotated by 5°, as shown in Figure 11. After rotation, no edge is exactly horizontal or vertical, allowing the staircase effect to be observed more clearly in the following experiments.

Point I.b: Then approximate Chapter 4's rule by supersampling: a pixel's value is the fraction of k × k sub positions inside it that the shape covers. The easy way to get that is to run your Part 2 fill on a canvas k times larger in each direction, with the shape scaled by k as well, and average each k × k block down to one pixel — the same computation, without testing sub-positions one at a time. Render the rotated star at k = 1, 2, 4 and 8, and show all four. k=8 is what II and III below mean by your largest k.

To approximate Chapter 4's rule that "Brightness is proportional to area of intersection," I used supersampling on the 5° rotated star. The final canvas size was 400 × 400 pixels. For a supersampling factor k, a pixel's value was determined by the fraction of its k × k sub-positions that were inside the shape.

The Part 2 non-zero winding fill was run on a canvas k times larger in each direction, with the rotated star scaled by k as well. Therefore, k = 1, 2, 4, and 8 used fill canvases of 400 × 400, 800 × 800, 1600 × 1600, and 3200 × 3200 pixels, respectively. Each k × k block was then averaged down to one pixel on the final 400 × 400 canvas.

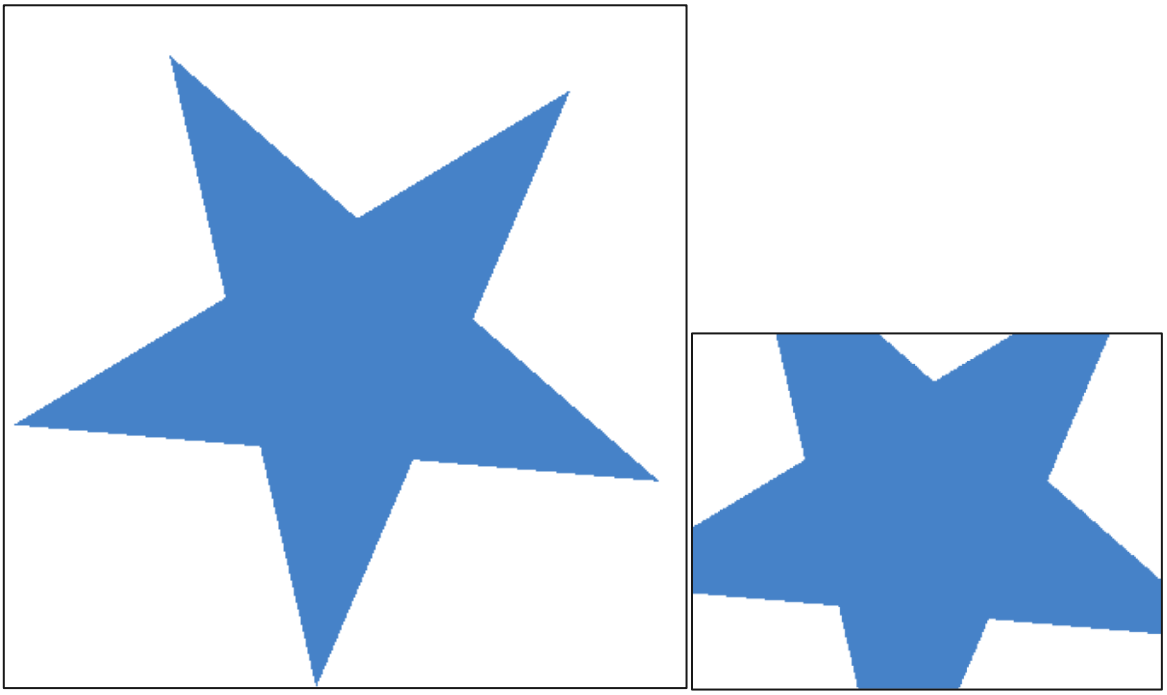


Figure 12: Rotated star rendered with supersampling at k = 1, using 1 sub-position per output pixel.

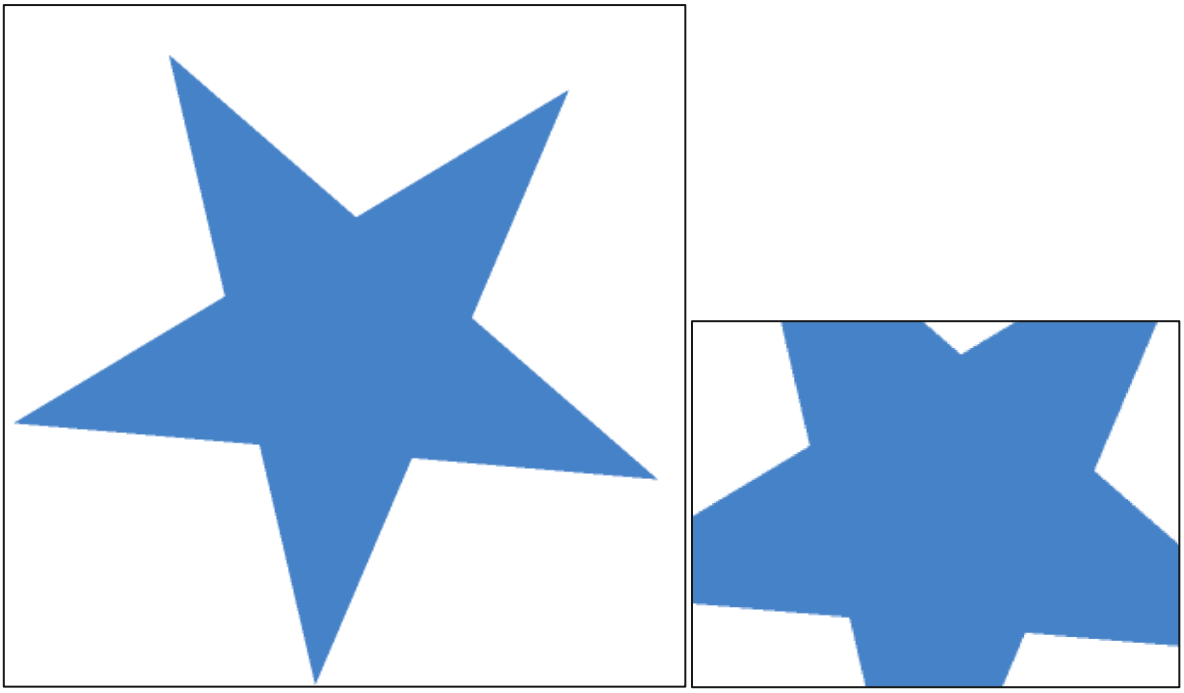


Figure 13: Rotated star rendered with supersampling at k = 2, using 4 sub-positions per output pixel.

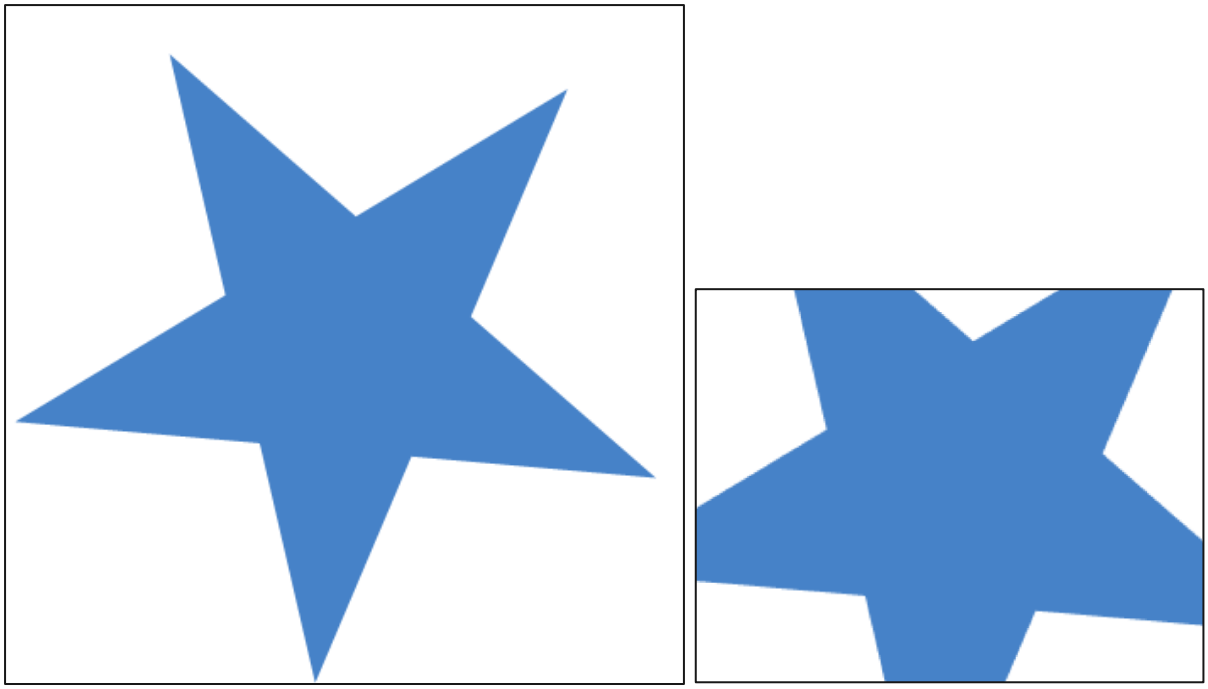


Figure 14. Rotated star rendered with supersampling at k = 4, using 16 sub-positions per output pixel.

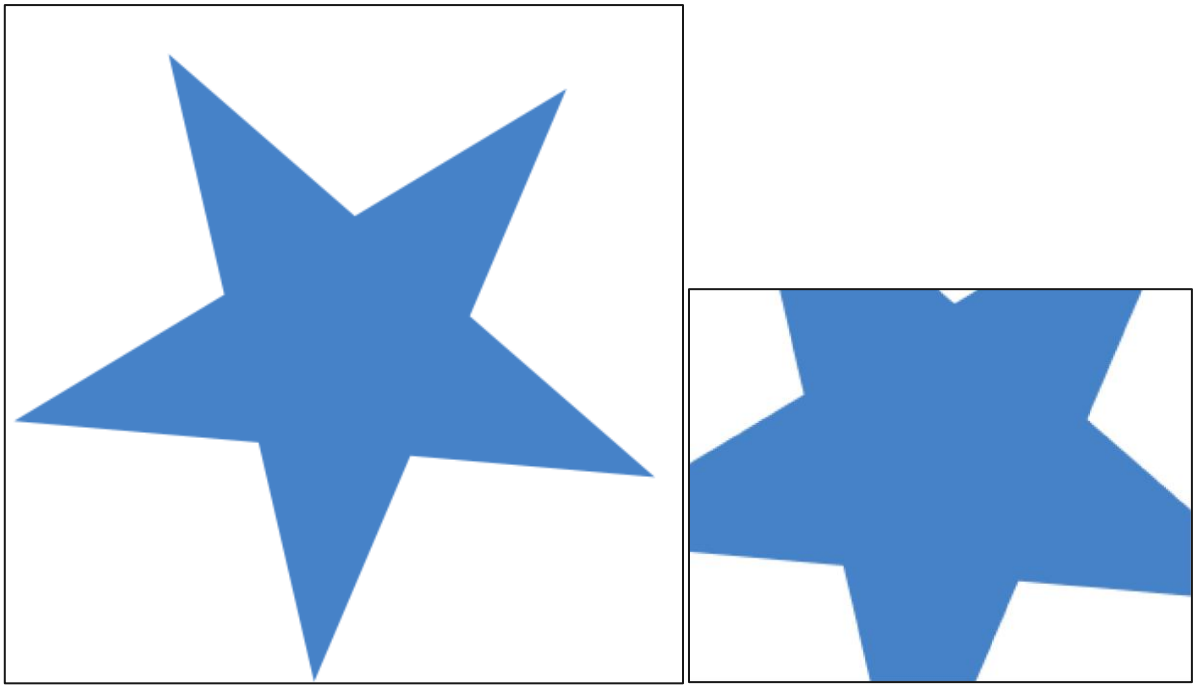


Figure 15: Rotated star rendered with supersampling at k = 8, using 64 sub-positions per output pixel.

Figures 12–15 show the rotated star rendered at k = 1, 2, 4, and 8. At k = 1, each output pixel contains only 1 sub-position, while k = 2, 4, and 8 use 4, 16, and 64 sub-positions per output pixel. As k increases, the fractional pixel values along the edges provide a closer approximation of the shape's area coverage. For the following parts, k = 8 is the largest k.

From the rendered results, the edges become progressively smoother as k increases from 1 to 8 because more sub-positions are used to approximate the area coverage of each output pixel. In the full-size images, the difference can appear subtle, so I also include slightly zoomed versions of the edge region. The zoomed views make the reduction of the staircase effect more visible, especially when comparing k = 1 with the larger k values.

II. Measure It (1.5 points)

Point II.a: Find the shallowest edge of your rotated star and pick a pixel row that crosses it near one of its ends — along the middle of that edge the star is filled on both sides, the core on one side and a spike on the other, so there is no transition there to plot. Plot the values along that row — scan along the edge, not across it. If other edges of the star cross the same row, plot only the stretch of the row around the edge you picked. Do this for k = 1 and for your largest k, and describe how the two plots differ in shape. (Scanning across the edge instead gives a transition one pixel wide at every k, so the two plots come out nearly the same.)

The shallowest edge of the rotated star was found by selecting the edge with the smallest absolute slope. The selected edge runs from (5.3194, 245.2045) to (383.8733, 278.3237), with an absolute slope of 0.087489. A pixel row at y = 250 was selected so that it crosses the shallowest edge near one of its ends. The selected edge crosses this row at approximately x = 60.132. Figure 16 shows the shallowest edge (red line), the selected pixel row (green line), and the local stretch used for plotting the pixel values (black line).

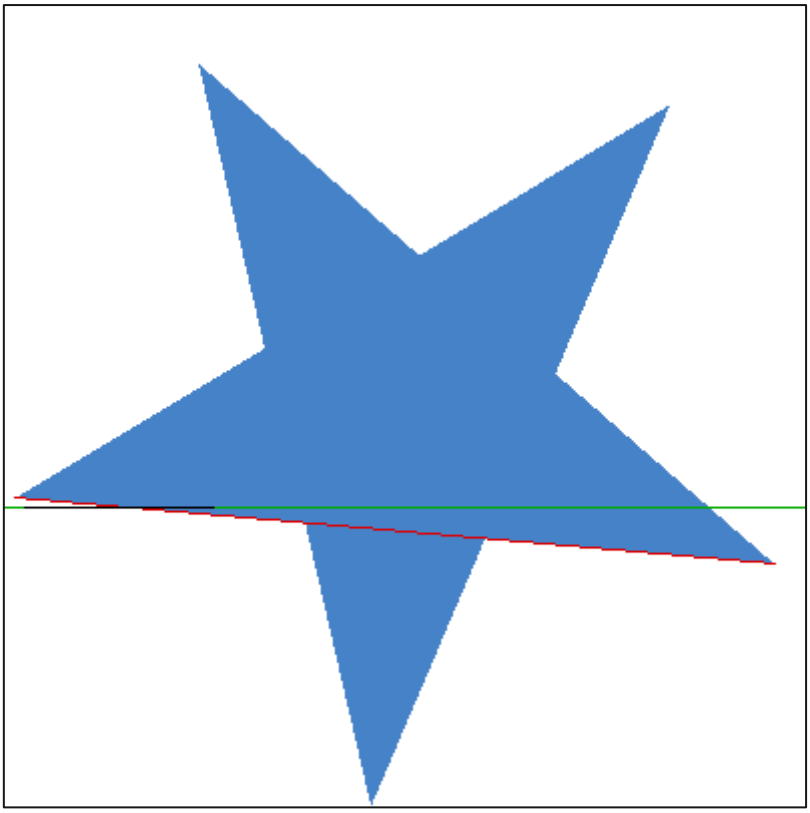


Figure 16: Rotated star showing the shallowest edge, the selected pixel row at y = 250, and the local stretch used for plotting the pixel values.

The pixel values were scanned along the selected row, following the shallow edge rather than scanning across it. Since other edges of the self-intersecting star can cross the same row, only the stretch from x = 10 to x = 104 around the selected edge was plotted. The same pixel row and stretch were used for both k = 1 and the largest k, k = 8.

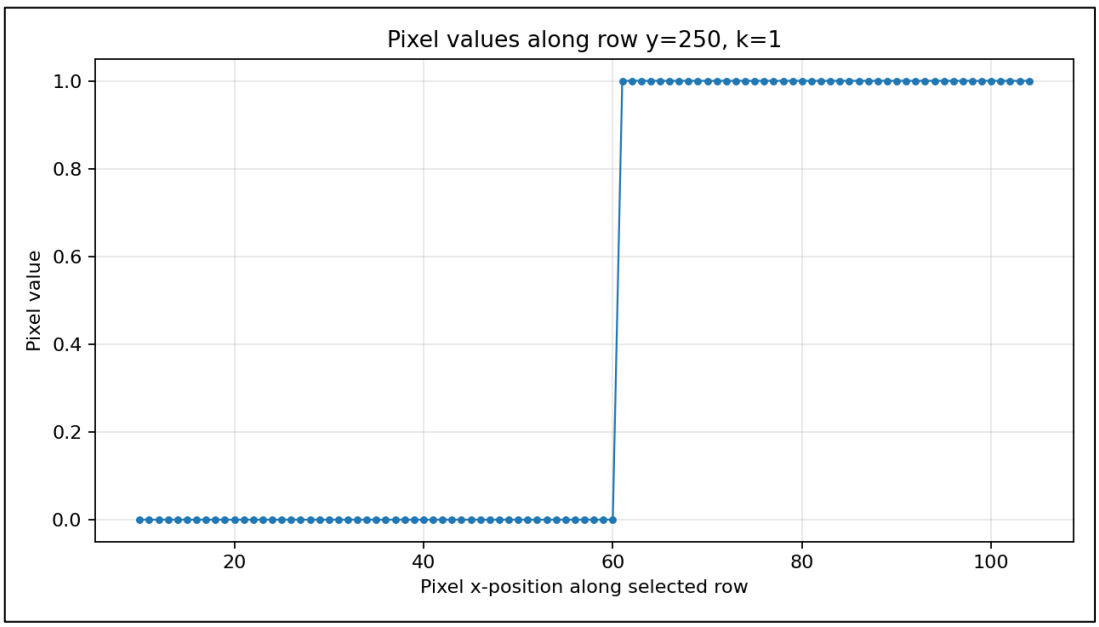


Figure 17: Pixel values along the selected row from x = 10 to x = 104 for k = 1. The transition from outside to inside is abrupt and contains only binary pixel values.

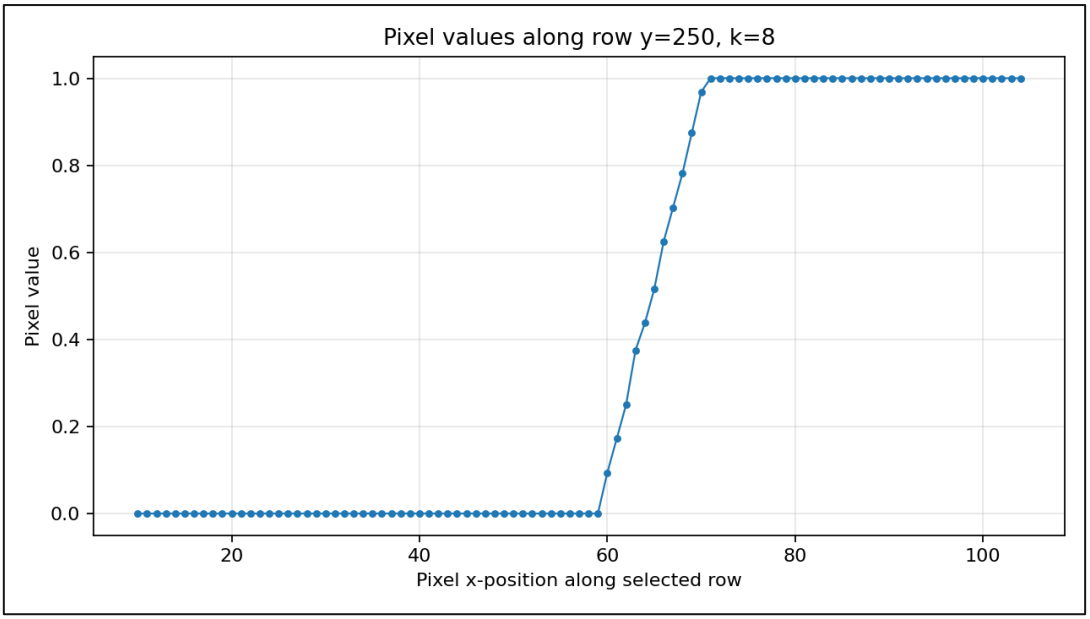


Figure 18: Pixel values along the same selected row from x = 10 to x = 104 for k = 8. The transition extends across several pixels and contains fractional pixel values.

Figure 17 shows the pixel values for k = 1. The plot contains only two distinct pixel values, 0 and 1, producing an abrupt transition between outside and inside. Figure 18 shows the same measurement for k = 8. In this case, 13 distinct pixel values occur, including fractional values, and the transition becomes more gradual across several pixels. Therefore, the k = 8 plot shows a more gradual transition shape than the k = 1 plot.

The measurement was taken along the edge because scanning across the edge would give a transition approximately one pixel wide at every k, causing the two plots to come out nearly the same.

Point II.b: Report the render time for each k, on one canvas size that you state. Say how the cost grows with k, and why.

K	Samples per output pixel	Supersampling canvas	Render time
1	1	400 × 400	0.103416 s
2	4	800 × 800	0.125069 s
3	16	1600 × 1600	0.208964 s
4	64	3200 × 3200	0.501134 s

Table 1: Render time for k = 1, 2, 4, and 8 using a final canvas size of 400 × 400 pixels.

The render time was measured for k = 1, 2, 4, and 8 using the same final canvas size of 400 × 400 pixels. As shown in Table 1, the measured render times were 0.103416 seconds for k = 1, 0.125069 seconds for k = 2, 0.208964 seconds for k = 4, and 0.501134 seconds for k = 8.

The render cost increases as k increases because the supersampling canvas is k times larger in each direction. Therefore, the number of high-resolution pixels processed grows with k × k. For example, k = 1 uses 1 sample per output pixel, while k = 8 uses 64 samples per output pixel. The measured render times do not increase by exactly the same factor as k × k because the total measured time also includes fixed overhead and implementation costs, but the increasing number of high-resolution pixels causes the overall render cost to grow with k.

III. Where the grid loses (2.5 points)

A wheel with 48 wedges was rendered on a 400 × 400 pixel canvas, with the center point at (200, 200) and a radius of 185 pixels. The wedges radiate from the center point and are separated by gaps as wide as the wedges. Each wedge is a triangle formed by the center point and two points on a circle, so the wheel consists of 48 sub-paths.

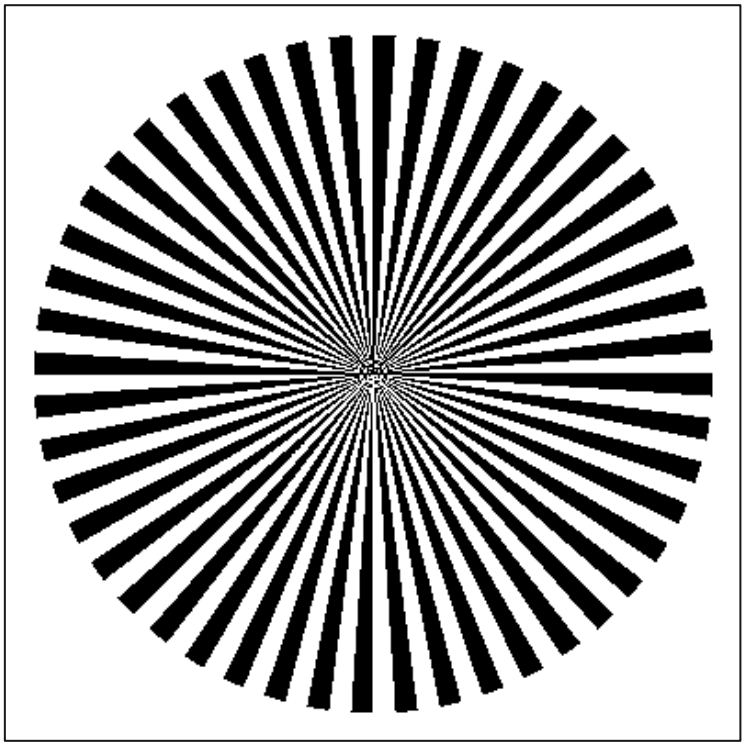


Figure 19: Wheel with 48 wedges (k = 1 on a 400 × 400 pixel canvas). The center visibly breaks down.

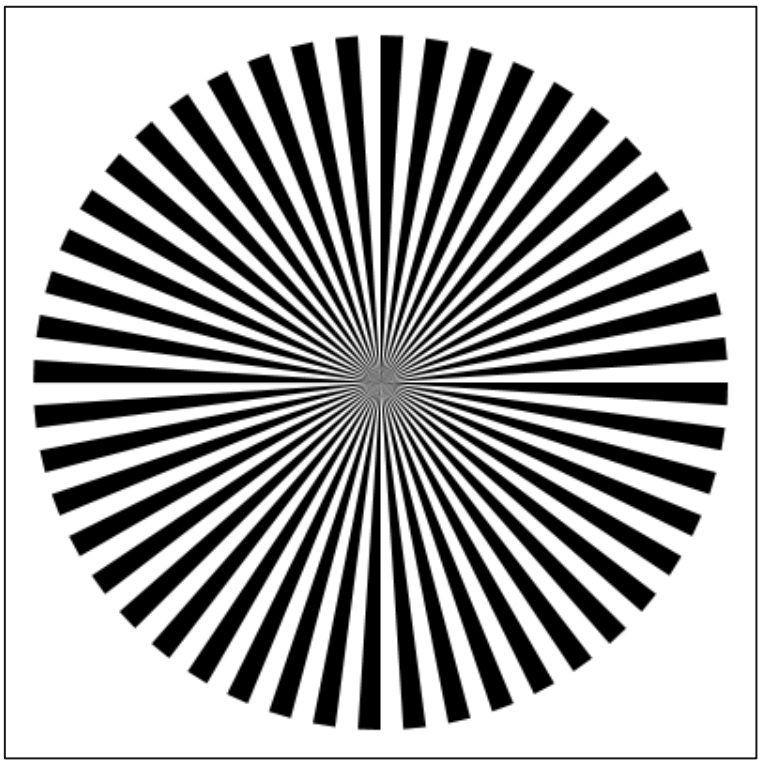


Figure 20: The same wheel with 48 wedges rendered at the largest k, k = 8, on a 400 × 400 pixel canvas.

The wheel was rendered at k = 1 and at the largest k, k = 8, as shown in Figure 19 and Figure 20. At k = 1, the center of the wheel visibly breaks down, so the 48 wedges are sufficient and no additional wedges are required.

Point 1: At k = 1 you will see something that is not merely jagged edges. Name it.

At k = 1, aliasing is visible near the center of the wheel. The regular wedge-gap pattern visibly breaks down and forms an incorrect pattern rather than preserving the individual wedges. Therefore, the artifact seen at k = 1 is aliasing, not merely jagged edges.

Point 2: Raising k cleans up the outside of the wheel, but however large k gets it never recovers the wedges at the center: what you named in (a) gives way to something closer to a uniform gray, never to the picture itself. Explain why, using Chapter 2: say how the spatial frequency behaves as you move toward the center, and what the Sampling Theorem therefore says about this picture.

Raising k from k = 1 to the largest k, k = 8, cleans up the outside of the wheel because the wedges and gaps are still wide enough there for the increased sampling to approximate their area coverage more accurately. However, the wedges at the center are not recovered.

As we move toward the center of the wheel, the wedges and gaps become narrower, so the black-and-white pattern changes over increasingly shorter spatial distances. Therefore, the spatial frequency increases toward the center. According to the Sampling Theorem, a frequency f can be properly reconstructed only when the sampling rate is at least 2f, the Nyquist rate. Near the center, the spatial frequency becomes too high for the final 400 × 400 pixel grid to satisfy this requirement.

Therefore, increasing k improves the approximation of the pixel area coverage, but it cannot recover the individual wedges at the center in the final image. The aliasing visible at k = 1 gives way to something closer to a uniform gray at k = 8, rather than recovering the picture itself.

Point 3: Chapter 2 also lists filters. Which kind is relevant here, and where in the process does the area coverage rule you implemented in I already apply it? Given your answer to (b), say what it does and does not achieve.

The relevant filter is a Low pass filter. Chapter 2 states that filters remove certain frequencies, and a Low pass filter is relevant here because the high spatial frequencies that cannot be represented correctly by the final pixel grid need to be reduced.

The area-coverage rule implemented in I already applies this during the averaging step. After the shape is rendered on a canvas k times larger in each direction, each k × k block is averaged down to one output pixel. At the largest k, k = 8, each output pixel therefore averages 64 sub-positions. This averaging reduces the effect of high spatial frequencies and produces intermediate gray values according to the area covered by the shape.

This Low pass filtering reduces aliasing and gives a cleaner representation of the outside of the wheel. However, it does not recover the individual wedges at the center. As explained in III.b, the spatial frequency there becomes too high for the final 400 × 400 pixel grid, so filtering gives something closer to a uniform gray rather than recovering the picture itself.

Part 4 : One Thing You Wanted to Know

Chosen Topic : Layers and Transparency

For Part 4, I will extend the renderer with partial-transparency compositing and study how opacity affects the rendered pixel values when layers are combined. The experiments will examine which pixels are affected by different opacity values, how opacity accumulates when the same shape is layered repeatedly, and how changing the order of overlapping layers affects the final pixel colors.

A. Effect of Opacity on Pixel Changes

Render one foreground shape over a fixed background using several opacity values, such as 0.25, 0.5, 0.75, and 1.0. Compare each result with the original background to identify exactly which pixels change and count how many pixels are affected. Then measure the RGB values of representative pixels inside the shape to observe how their colors change as opacity increases. This experiment examines both where transparency affects the image and how strongly it changes the pixel values.

B. Opacity Accumulation

Render the same partially transparent shape repeatedly at the same position using a fixed opacity value. Measure the effective opacity after 1, 2, 4, and 8 overlapping layers, then determine whether the increase is linear and describe the pattern followed by the measurements.

C. Effect of Layer Order

Render two overlapping shapes with different colors but the same opacity. Compare the result of placing Shape A over Shape B with placing Shape B over Shape A. Measure the pixel differences and determine whether changing only the layer order changes the final rendered colors.

Point A: Effect of Opacity on Pixel Changes

A filled circle was rendered over a fixed white background on a 400 × 400 pixel canvas. The circle was centered at (200, 200) with a radius of 150 pixels. Its foreground RGB color was (0, 100, 220), while the background RGB was (255, 255, 255). The circle was constructed using the same method as in Part 2. It was rendered at opacity values 0.00, 0.25, 0.50, 0.75, and 1.00, where opacity 0.00 was included as a fully transparent baseline.

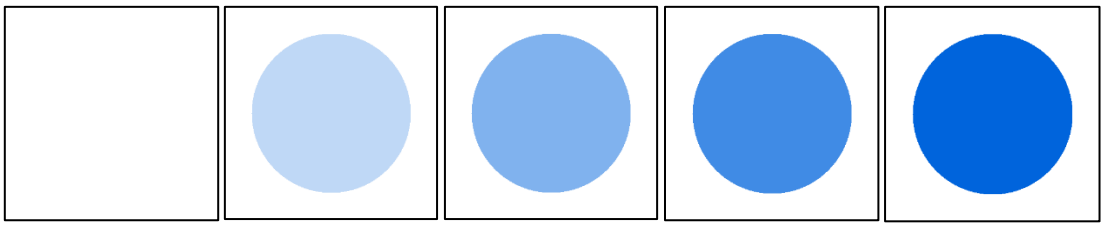


Figure 21: Foreground circle rendered over a fixed background at opacity 0.00, 0.25, 0.50, 0.75, and 1.00, from left to right.

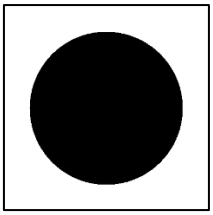


Figure 22: Changed-pixel mask for opacity 0.25–1.00, showing the same 70,678 circle pixels that changed from the background.

Each rendered result was compared with the original background to identify exactly which pixels changed and count how many pixels were affected. At opacity 0.00, the number of changed pixels was 0. For opacity values 0.25, 0.50, 0.75, and 1.00, 70,678 pixels changed in every case. This number is equal to the number of pixels occupied by the rasterized circle, meaning that every pixel belonging to the circle was affected at each non-zero opacity. The same set of circle pixels changed because the shape and its position remained fixed; only the strength of the color change varied with opacity.

Opacity Measurement Summary

Opacity	Changed Pixels	Center RGB	Color Change (%)
0.00	0	(255, 255, 255)	0.00%
0.25	70678	(191, 216, 246)	25.12%
0.50	70678	(128, 178, 238)	49.75%
0.75	70678	(64, 139, 229)	74.88%
1.00	70678	(0, 100, 220)	100.00%

Figure 23: Opacity Measurement Summary showing the changed pixel count, center RGB value, and color change percentage for each opacity level.

The center pixel at (200, 200) was used as the representative pixel inside the shape. Its RGB values were (255, 255, 255), (191, 216, 246), (128, 178, 238), (64, 139, 229), and (0, 100, 220) at opacity values 0.00, 0.25, 0.50, 0.75, and 1.00, respectively. The corresponding color change percentages were 0.00%, 25.12%, 49.75%, 74.88%, and 100.00%.

As opacity increased, the center pixel moved progressively from the background RGB toward the foreground RGB. The color change percentage follows the opacity value closely because alpha blending computes the output color as a weighted combination of the background and foreground colors. Small differences from the exact values of 25%, 50%, and 75% are caused by rounding the computed RGB channel values to integers. These measurements show how strongly transparency changes the pixel values as opacity increases.

Point B: **Opacity Accumulation**

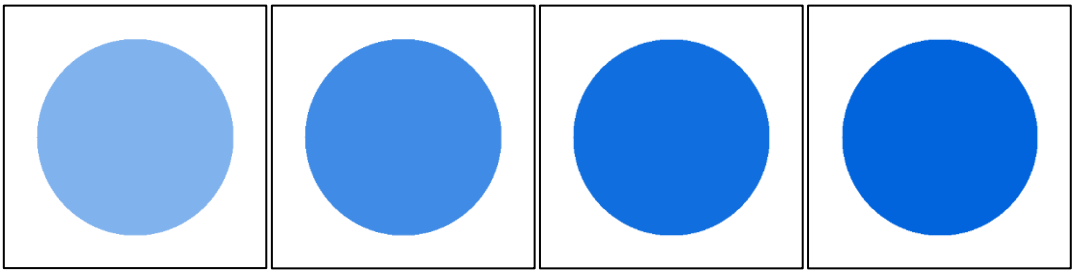


Figure 24: The same circle rendered with 1, 2, 4, and 8 overlapping layers at a fixed opacity of 0.50 per layer, from left to right.

The same partially transparent circle was rendered repeatedly at the same position using a fixed opacity of 0.50 per layer. The circle was composited over the same white background for 1, 2, 4, and 8 overlapping layers. The measured effective opacity was 49.75% after 1 layer, 74.88% after 2 layers, 93.69% after 4 layers, and 99.72% after 8 layers.

Opacity Accumulation Summary

Layers	Center RGB	Effective Opacity (%)
1	(128, 178, 238)	49.75%
2	(64, 139, 229)	74.88%
4	(16, 110, 222)	93.69%
8	(1, 100, 220)	99.72%

Figure 25: Opacity Accumulation Summary showing the center RGB and measured effective opacity after 1, 2, 4, and 8 overlapping layers.

The increase is not linear. A 0.50 opacity layer added on top of another 0.50 opacity layer does not simply give 1.00 opacity. The second layer is blended with the result of the first layer, so only the remaining background contribution is reduced again. Conceptually, after the first 0.50 layer, 50% of the background contribution remains. Applying another 0.50 layer removes half of that remaining 50%, leaving 25% of the background contribution. Therefore, the effective opacity becomes about 75%, not 100%.

As more layers are added, the remaining background contribution keeps being reduced by the same proportion, so the effective opacity approaches 100% but does not increase by a fixed amount each time.

Point C: Effect of Layer Order

Two overlapping circles with different colors but the same opacity of 0.50 were rendered on a 400 × 400 pixel canvas. Shape A had RGB (220, 70, 70) and was centered at (140, 200), while Shape B had RGB (50, 120, 220) and was centered at (260, 200). Both circles had a radius of 120 pixels.

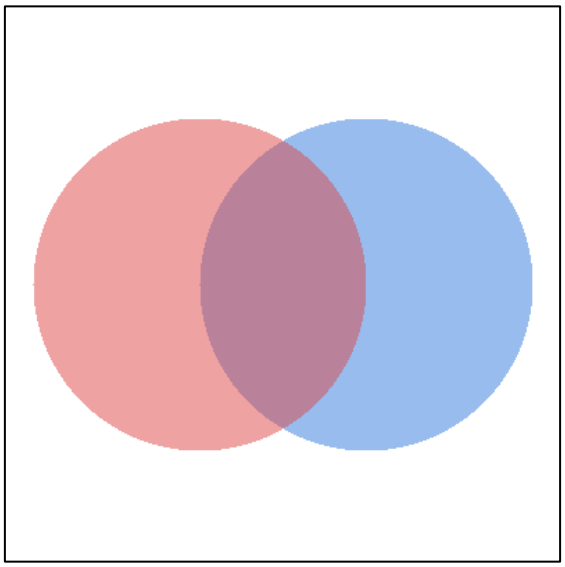


Figure 26: Shape A over Shape B, with both shapes rendered at opacity 0.50.

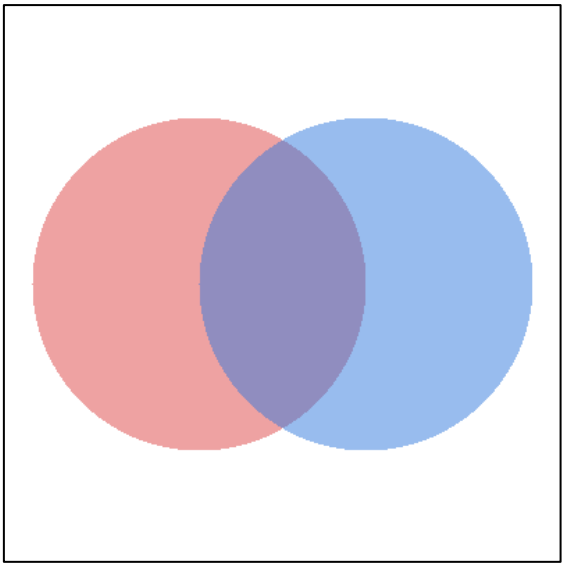


Figure 27: Shape B over Shape A, with both shapes rendered at opacity 0.50.

The result of placing Shape A over Shape B was compared with placing Shape B over Shape A. The two shapes had 17,686 overlap pixels, and all 17,686 of these pixels changed when only the layer order was reversed. At the representative overlap pixel (200, 200), the final RGB was (186, 129, 154) for Shape A over Shape B and (144, 141, 191) for Shape B over Shape A.

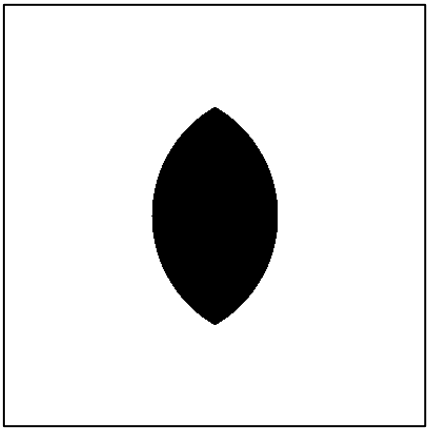


Figure 28: Pixel-difference mask between the two layer orders. The black region shows the 17,686 overlap pixels whose final RGB values changed when the layer order was reversed.

Therefore, changing only the layer order changes the final rendered colors in the overlap area. Pixels outside the overlap remain unchanged because only one shape contributes to those pixels, while pixels inside the overlap depend on which partially transparent shape is composited last.

Even when the shapes, colors, positions, and opacity are unchanged, reversing the layer order produces different final colors in the overlap area. This happens because alpha compositing is order-dependent: the shape composited last is blended over the result of the previous layer, so it has a different contribution to the final pixel color.